

Project: CustomerCare360 - Utility Customer Information & Billing Management System

1. Introduction

CustomerCare360 is a web-based system designed for energy and utilities providers (electricity, gas, water) to manage customer lifecycle, service contracts, billing cycles, complaints, service requests, and customer engagement. The system supports customer onboarding, account and premise management, service start/stop, billing presentation, dispute handling, and customer analytics.

It uses a **REST API-based backend architecture** and **Angular/React** for the frontend.

Compatible with **Java (Spring Boot)** frameworks and can also be adapted to **.NET (ASP.NET Core)**.

Actors / Users

- **Utility Customer:** Manages profile, views bills, raises service requests and complaints, tracks status.
- **Customer Service Agent:** Handles inquiries, complaints, payments references, service orders.
- **Billing Operations Analyst:** Monitors billing cycles, adjustments, exceptions, and summaries.
- **Field Service Coordinator:** Manages customer-related service orders (connect/disconnect/move).
- **Utilities Admin:** Configures customer classes, service types, billing cycles, user roles, and reports.

2. Module Overview

2.1 Identity & Customer Access Management

Handles registration, authentication, RBAC, and customer self-service access.

2.2 Customer, Account & Premise Management

Defines customers, service accounts, premises, and service agreements.

2.3 Service Orders & Requests

Manages customer service requests and field service orders (connect/disconnect/maintenance).

2.4 Billing Cycle & Bill Presentation

Handles billing periods, bill generation summaries, and bill views (no payment gateways in Phase 1).

2.5 Adjustments, Disputes & Complaints

Manages billing disputes, adjustments, complaints, and resolutions.

2.6 Customer Analytics & Service KPIs

Generates dashboards for billing accuracy, complaint resolution SLA, churn indicators.

2.7 Notifications & Communications

Sends in-app alerts for bill availability, service order updates, and complaint status.

3. Architecture Overview

- **Frontend:** Angular or React
- **Backend:** REST API-based architecture
- **Database:** Relational Database (MySQL/PostgreSQL/SQL Server)

4. Module-Wise Design

4.1 Identity & Customer Access Management Module

Features:

- Register customers/agents/admins
- Login, RBAC, session management

Entities:

- **User**
 - UserID
 - Name
 - Role (Customer/Agent/Billing/Admin)
 - Email
 - Phone

4.2 Customer, Account & Premise Management Module

Features:

- Maintain customer profile, service accounts, premises, and agreements

Entities:

- **Customer**
 - CustomerID
 - UserID
 - CustomerType (Residential/Commercial/Industrial)
 - ContactInfo
 - Status (Active/Inactive)

- **ServiceAccount**
 - AccountID
 - CustomerID
 - ServiceType (Electric/Gas/Water)
 - StartDate
 - EndDate
 - Status (Active/Suspended/Closed)
- **Premise**
 - PremiseID
 - Address
 - Region
 - MeterID
 - Status (Energized/Disconnected)

4.3 Service Orders & Requests Module

Features:

- Capture customer requests; manage field service orders

Entities:

- **ServiceRequest**
 - RequestID
 - CustomerID
 - RequestType (NewConnection/Disconnect/Complaint/Inquiry)
 - CreatedDate
 - Priority
 - Status (Open/InProgress/Resolved/Closed)
- **ServiceOrder**
 - OrderID
 - ServiceAccountID
 - PremiseID
 - OrderType (Connect/Disconnect/Inspection)
 - ScheduledDate
 - CompletionDate

- Status (Scheduled/InProgress/Completed/Failed)

4.4 Billing Cycle & Bill Presentation Module

Features:

- Manage billing cycles, summarize charges, present bills

Entities:

- **BillingCycle**
 - CycleID
 - ServiceType
 - PeriodStart
 - PeriodEnd
 - BillDate
 - Status (Open/Completed)
- **Bill**
 - BillID
 - AccountID
 - CycleID
 - Usage
 - Amount
 - DueDate
 - Status (Generated/Adjusted/Closed)

4.5 Adjustments, Disputes & Complaints Module

Features:

- Handle disputes, adjustments, and complaint resolution

Entities:

- **Adjustment**
 - AdjustmentID
 - BillID
 - Reason
 - AmountDelta
 - ApprovedBy
 - Status (Requested/Approved/Rejected)

- **Complaint**
 - ComplaintID
 - CustomerID
 - Category (Billing/Service/Outage)
 - Description
 - LoggedDate
 - Status (Open/InProgress/Resolved/Closed)

4.6 Customer Analytics & Service KPIs Module

Features:

- SLA tracking, billing accuracy, customer churn indicators

Entities:

- **CustomerReport**
 - ReportID
 - Scope (Region/ServiceType/Period)
 - Metrics (AvgResolutionTime, BillAdjustmentsRate, ComplaintVolume)
 - GeneratedDate

4.7 Notifications & Communications Module

Features:

- Notify customers about bills, service orders, complaints

Entities:

- **Notification**
 - NotificationID
 - UserID
 - Message
 - Category (Bill/Service/Complaint)
 - Status (Unread/Read/Dismissed)
 - CreatedDate

5. Deployment Strategy

- **Local:** Angular/React frontend, Spring Boot/.NET Core backend, local DB.

- **Production:** Cloud or on-prem deployment with secure APIs (self-contained, no external integrations).

6. Database Design

Tables:

- User(UserID, Name, Role, Email, Phone)
- Customer(CustomerID, UserID, CustomerType, ContactInfo, Status)
- ServiceAccount(AccountID, CustomerID, ServiceType, StartDate, EndDate, Status)
- Premise(PremiseID, Address, Region, MeterID, Status)
- ServiceRequest(RequestID, CustomerID, RequestType, CreatedDate, Priority, Status)
- ServiceOrder(OrderID, ServiceAccountID, PremiseID, OrderType, ScheduledDate, CompletionDate, Status)
- BillingCycle(CycleID, ServiceType, PeriodStart, PeriodEnd, BillDate, Status)
- Bill(BillID, AccountID, CycleID, Usage, Amount, DueDate, Status)
- Adjustment(AdjustmentID, BillID, Reason, AmountDelta, ApprovedBy, Status)
- Complaint(ComplaintID, CustomerID, Category, Description, LoggedDate, Status)
- CustomerReport(ReportID, Scope, Metrics, GeneratedDate)
- Notification(NotificationID, UserID, Message, Category, Status, CreatedDate)

7. User Interface Design

- **Customer Portal:** View bills, service requests, complaints, notifications.
- **Agent Dashboard:** Customer search, requests/complaints queue, adjustments handling.
- **Billing Ops Dashboard:** Billing cycles, exceptions, adjustment trends.
- **Admin Dashboard:** Configure customer classes, services, cycles, policies, reports.

8. Non-Functional Requirements

- **Performance:** Handle **50,000 concurrent users** during bill release cycles.
- **Security:** Encrypted data, RBAC, audit trails.
- **Scalability:** Support large customer bases across multiple regions.
- **Availability:** Target **99.9% uptime**.
- **Maintainability:** Modular services, API versioning, automated migrations.
- **Observability:** Centralized logs and service KPIs.

9. Assumptions & Constraints

- Focus on customer information and billing reference management only.
- No external payment systems or CRM integrations in Phase 1.
- All notifications are in-app only.
- Implementable using standard Java Spring Boot, Angular/React, and relational DB tooling.